1 Lecture 11 (Oct 1) - Divide and Conquer: Quicksort and Linear Time Selection

1.1 Divide and Conquer

Mergesort is an example of a **divide and conquer** algorithm. There are no hard rules for what counts as divide and conquer, but the high-level idea is:

1. **Divide** the problem into smaller instances of the same problem
2. **Delegate** the smaller instances to the recursion fairy
3. **Combine** the solutions of the subproblems into a solution for the original instance. This step usually needs some extra work.

For mergesort: divide into two halves of size about $\frac{n}{2}$, sort both recursively, then combine with `Merge`. Combining two sorted arrays is much easier than sorting from scratch.

To analyze a divide and conquer algorithm, write a **recurrence** for the running time in terms of the smaller instances, then solve it (usually with a recursion tree).

1.2 Quicksort

Tony Hoare (1961).

1. Choose a **pivot** element. Any element is allowed; the choice is arbitrary.
2. **Partition** the array in linear time: everything smaller than the pivot moves to its left, everything bigger moves to its right.
3. Now the pivot is in its final position, but the two sides are not sorted. Neither side contains the pivot, so both are strictly smaller. Recursively sort the left side, then recursively sort the right side.

```
QuickSort(A[1..n]):
  if n > 1
    choose a pivot element A[p]
    r ← Partition(A, p)
    QuickSort(A[1..r - 1])
    QuickSort(A[r + 1..n])
```

1.2.1 Rank

`Partition` returns r, the new index of the pivot. This is the **rank** of the pivot:

- $\text{rank}(x) = 1 + (\text{number of elements smaller than } x)$
- Equivalently, the position of x in sorted order (1-indexed)

After partitioning, the elements smaller than the pivot are in $A[1..r-1]$ and the elements larger are in $A[r+1..n]$. Duplicate elements cause no problem: take the rank to mean the pivot's final position.

1.2.2 Partition

For completeness only. The correctness argument is subtle; see the lecture notes and Erickson's book. Move the pivot to the end, out of the way. Walk through the array; whenever an element is smaller than the pivot, swap it into the left part. Finally, swap the pivot into place.

```
Partition(A[1..n], p):
  swap A[p] ↔ A[n]
  ℓ ← 0                          ⟨⟨#items < pivot⟩⟩
  for i ← 1 to n - 1
    if A[i] < A[n]
      ℓ ← ℓ + 1
      swap A[ℓ] ↔ A[i]
  swap A[n] ↔ A[ℓ + 1]
  return ℓ + 1
```

Partitioning takes $O(n)$ time.

1.2.3 Worst case

The recursive calls do **not** have to be on halves. With pivot rank r:

$$T(n) = T(r-1) + T(n-r) + O(n)$$

If the pivot is the smallest (or largest) element, one call is empty and the other has $n-1$ elements:

$$T(n) = T(n-1) + O(n)$$

The recursion tree is a long path:

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{n-1}$	$n-1$
2	$\boxed{n-2}$	$n-2$
⋮	$\boxed{\ldots}$	⋮
$n-1$	$\boxed{1}$	1

This is an **arithmetic series**, not a geometric one. Its sum is **not** proportional to the largest term. It is proportional to (largest term) × (number of terms):

$$T(n) = \sum_{i=1}^{n} i = \boxed{O(n^2)}$$

Every simple deterministic pivot rule has bad inputs that force this: the first element, the last element, even the median of the first, middle and last elements.

1.2.4 Worst-case analysis

This course uses worst-case analysis: A stated running time must hold for every input. So all we can promise for quicksort is $O(n^2)$.

A loose bound such as $O(n!)$ is technically correct but useless, and is marked wrong. Give the bound that naturally falls out of your analysis technique.

1.2.5 Good pivots

Quicksort is fast in practice: unless someone gives it nasty inputs on purpose, the pivot rank is usually near $\frac{n}{2}$. With probability about $\frac{1}{3}$, the pivot is in the **middle third**, so both calls have at most $2n/3$ elements. In the most unbalanced such case:

$$T(n) \leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + O(n)$$

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{3}}$ $\boxed{\frac{2n}{3}}$	n
2	$\boxed{\frac{n}{9}}$ $\boxed{\frac{2n}{9}}$ $\boxed{\frac{2n}{9}}$ $\boxed{\frac{4n}{9}}$	n
$\vdots$	$\boxed{\ldots}$	$\leq n$

The tree is unbalanced, but every level sums to (at most) n. Some branches reach their base cases early, so lower levels may be missing nodes; that is fine for an upper bound.

The longest path follows the larger calls, multiplying by $\frac{2}{3}$ each level. The number of times you can multiply n by $\frac{2}{3}$ before reaching 1 is the number of times you can divide by $\frac{3}{2}$:

$$\text{levels} \leq \log_{3/2} n = O(\log n)$$

Any constant log base changes the result only by a constant factor, so:

$$T(n) = n \cdot \log_{3/2} n = \boxed{O(n \log n)}$$

1.3 Selection

Median: given $A[1..n]$, find the element of rank $\lceil \frac{n}{2} \rceil$.

1.3.1 Sort first

Mergesort the array and return $A[\lceil \frac{n}{2} \rceil]$. This takes $O(n \log n)$ time, and is a perfectly valid answer on a homework or exam. **If you can use something from class as a black box, do so.** But we only want one element, so try to do better.

1.3.2 Recursive idea

Pick a pivot and partition. Compare its rank r with $\lceil \frac{n}{2} \rceil$:

- $r < \lceil \frac{n}{2} \rceil$: everything left of the pivot is even smaller than the median, so the median is to the **right**. Recurse on the right side.
- $r > \lceil \frac{n}{2} \rceil$: the median is to the **left**. Recurse on the left side.
- $r = \lceil \frac{n}{2} \rceil$: the pivot is the median.

Do not pick another pivot inside the right side yourself. That is doing the recursion fairy's job, and it makes them grumpy. Just recurse.

Technical issue: inside the right subarray, the median no longer has rank $\lceil \frac{n}{2} \rceil$. Its rank is smaller relative to the subarray. So the original problem is not general enough to recurse on. Sometimes you need to solve a **more general problem** that you can call recursively.

1.3.3 The selection problem

Given an array $A[1..n]$ and an integer k with $1 \leq k \leq n$, find the element of rank k.

1.4 Quickselect

Tony Hoare, on the same page of the paper as quicksort. Think of it as a **one-armed quicksort**.

```
QuickSelect(A[1..n], k):
  if n = 1
    return A[1]
  else
    choose a pivot element A[p]
    r ← Partition(A[1..n], p)
    if k < r
      return QuickSelect(A[1..r - 1], k)
    else if k > r
      return QuickSelect(A[r + 1..n], k - r)
    else
      return A[r]
```

- $k < r$: everything right of the pivot is too big. Search the left side, still for rank k.
- $k > r$: the target is bigger than the pivot. Search the right side. The right side is missing the r smallest elements, so look for rank $k - r$.
- $k = r$: the pivot is the answer.

1.4.1 Running time

Worst case: the pivot is always the smallest or largest element. Same path-shaped recursion tree as quicksort, so $O(n^2)$.

With a pivot in the middle third, there is only **one** recursive call, on at most $2n/3$ elements:

$$T(n) \leq T\left(\frac{2n}{3}\right) + O(n)$$

Level	**Nodes**	**Sum**
0	$\boxed{n}$	n
1	$\boxed{\frac{2n}{3}}$	$\frac{2n}{3}$
2	$\boxed{\frac{4n}{9}}$	$\frac{4n}{9}$
i	$\boxed{\left(\frac{2}{3}\right)^i n}$	$\left(\frac{2}{3}\right)^i n$

The level sums form a **decreasing geometric series**. A geometric series whose ratio is not 1 is proportional to its largest term, here the root. So $T(n) = O(n)$, **if** we can always find a pivot near the middle.

1.5 Median of Medians Selection

Blum, Floyd, Pratt, Rivest and Tarjan (1970). Idea: take a representative **sample** of the array, find the sample's median recursively, and use it as the pivot. The sample's median should be close to the real median.

```
MomSelect(A[1..n], k):
  if n ≤ 25
    use brute force
  else
    m ← ⌈n/5⌉
    for i ← 1 to m
      M[i] ← MedianOfFive(A[5i − 4..5i])
    mom ← MomSelect(M[1..m], ⌊m/2⌋)
    r ← Partition(A[1..n], mom)
    if k < r
      return MomSelect(A[1..r − 1], k)
    else if k > r
      return MomSelect(A[r + 1..n], k − r)
    else
      return mom
```

1. **Big base case**: if $n \leq 25$, sort and pick directly. Constant time.
2. Split the array into blocks of 5. Find the median of each block and put it in array M of size about $\frac{n}{5}$.

3. **Recursively** find the median of M, the **median of medians** (mom). This is allowed because $\frac{n}{5} < n$.
4. Run quickselect with mom as the pivot.

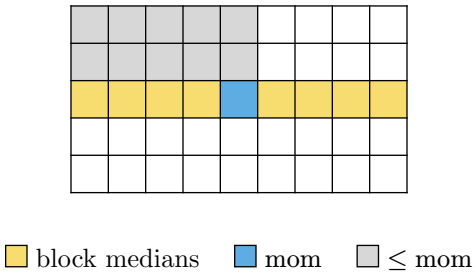
You can think of step 3 as an **approximate median fairy**, which is secretly the recursion fairy in disguise.

1.5.1 Analysis

Work outside the recursive calls: each median of five takes constant time, and there are $\frac{n}{5}$ of them; partitioning takes $O(n)$. So $O(n)$ total.

The question: is mom a good pivot? Show that neither recursive call is too big. **For the analysis only** (the algorithm does not do this), arrange A into a $5 \times \frac{n}{5}$ grid:

1. Each column is one block of 5.
2. Sort each column, smallest on top. The block medians land in the middle row.
3. Sort the columns by their medians. mom is now in the center.



- Each median to the left of mom is smaller than mom.
- Each column is sorted, so the elements above those medians are smaller still.
- There are $\frac{n}{5}$ columns, so about $\frac{n}{10}$ columns are left of mom. Each has 3 elements (the median and the two above it) smaller than mom.

So at least $3n/10$ elements are smaller than mom, and **at most $7n/10$ elements are larger**. Symmetrically, at most $7n/10$ elements are smaller. The other cells could go either way, but they are not needed. Either recursive call has at most $7n/10$ elements:

$$T(n) \leq O(n) + T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right)$$

Level	Nodes	Sum
0	$\boxed{n}$	n
1	$\boxed{\frac{n}{5}}$ $\boxed{\frac{7n}{10}}$	$\frac{9n}{10}$
2	$\boxed{\frac{n}{25}}$ $\boxed{\frac{7n}{50}}$ $\boxed{\frac{7n}{50}}$ $\boxed{\frac{49n}{100}}$	$\frac{81n}{100}$
i	$\boxed{\ldots}$	$\left(\frac{9}{10}\right)^i n$

$\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, so the level sums form a **decreasing geometric series**. The number of levels does not matter; the sum is proportional to the root:

$$T(n) = \boxed{O(n)}$$

This is optimal: any algorithm must look at every element.

1.5.2 Why blocks of 5?

5 is the smallest odd block size for which the analysis works (7 also works). With blocks of 3, the same argument gives

$$T(n) \leq O(n) + T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right)$$

Every level sums to n, which is the good-pivot quicksort tree: $O(n \log n)$, no better than sorting.

1.5.3 Takeaways

- Divide and conquer can use **unexpected** subproblems, such as recursing on a sample just to find a pivot.
- Subproblems can be **unbalanced**. Use recursion trees to analyze them.

1.6 Preview: Fast Multiplication

Next lecture: multiply two n-digit numbers with $O\left(n^{\log_2 3}\right) \approx O\left(n^{1.59}\right)$ single-digit multiplications. Another divide and conquer algorithm, which splits each number's digits into two halves.